

Symbolic & Concolic Execution

Symbolic & Concolic Execution

Task 1 - Symbolic Execution with Loops

Consider the function gcd below, which computes the greatest common divisor of two natural numbers using the Euclidean algorithm. The function must be invoked with strictly positive arguments. In this task, you will apply symbolic execution to gcd

```
// requires: a > 0 && b > 0
int gcd(int a, int b) {
    while(b != 0) {
        int tmp = b;
        b = a mod b;
        a = tmp;
    }
    return a;
}
```

a) Symbolic execution cannot handle unbounded loops such as in gcd. We require static bound and need to explicitly unroll the loop. Apply idea on gcd and obtain bounded_gcd where loop body from gcd is executed **at most** twice.

Answer

```
// requires: a > 0 && b > 0
int bounded_gcd(int a, int b) {
    int c = 0;
    while (b != 0 && c < 2) {
        int tmp = b;
        b = a mod b;
        a = tmp;
        c = c + 1;
    }
    return a;
}
```

b) Perform symbolic execution on bounded_gcd to find all symbolic states with a satisfiable path constraint at return statement. Incorporate precondition in your analysis.

Answer State 0 int bounded_gcd(int a, int b)

- σ :
 - $a \rightarrow A_0$
 - $b \rightarrow B_0$
- π :

State 1 int c = 0;

- σ :
 - $a \rightarrow A_0$
 - $b \rightarrow B_0$
 - $c \rightarrow 0$
- π :

State 2 **while** (b!=0 && c<=2)

- σ :
 - $a \rightarrow A_0$
 - $b \rightarrow B_0$
 - $c \rightarrow 0$
- π : $B_0 \neq 0 \wedge c \leq 2$
- No branching because pre-condition $B_0 > 0$ ensures $B_0 \neq 0$

State 3 **int tmp = b;**

- σ :
 - $a \rightarrow A_0$
 - $b \rightarrow B_0$
 - $c \rightarrow 0$
 - $t \rightarrow B_0$
- π :

State 4 **b = a mod b;**

- σ :
 - $a \rightarrow A_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 0$
 - $t \rightarrow B_0$
- π :

State 5 **a = tmp;**

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 0$
 - $t \rightarrow B_0$
- π :

State 6 **c = c+1;**

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 1$
 - $t \rightarrow B_0$
- π :

State 7 **while** (b != 0 && c <= 2){ (Return)

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 1$
- π : $(A_0 \bmod B_0) = 0 \wedge c \leq 2$
- Pre-condition $(A_0 > 0, B_0 > 0)$ does not prevent $(A_0 \bmod B_0) = 0$, therefore we have two branches!

State 8 **while** (b != 0 && c <= 2){ (Continue)

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 1$
- π : $(A_0 \bmod B_0) \neq 0 \wedge c \leq 2$

State 9 `int tmp = b;`

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (A_0 \bmod B_0)$
 - $c \rightarrow 1$
 - $t \rightarrow (A_0 \bmod B_0)$
- π :

State 10 `b = a mod b;`

- σ :
 - $a \rightarrow B_0$
 - $b \rightarrow (B_0 \bmod (A_0 \bmod B_0))$
 - $c \rightarrow 1$
 - $t \rightarrow (A_0 \bmod B_0)$
- π :

State 11 `a = tmp;`

- σ :
 - $a \rightarrow (A_0 \bmod B_0)$
 - $b \rightarrow (B_0 \bmod (A_0 \bmod B_0))$
 - $c \rightarrow 1$
 - $t \rightarrow (A_0 \bmod B_0)$
- π :

State 12 `c = c + 1`

- σ :
 - $a \rightarrow (A_0 \bmod B_0)$
 - $b \rightarrow (B_0 \bmod (A_0 \bmod B_0))$
 - $c \rightarrow 2$
 - $t \rightarrow (A_0 \bmod B_0)$
- π :

State 13 `while (b != 0 && c <= 2){ (End)`

- σ :
 - $a \rightarrow (A_0 \bmod B_0)$
 - $b \rightarrow (B_0 \bmod (A_0 \bmod B_0))$
 - $c \rightarrow 2$
- π : $c \not\leq 2$
- No need to branch
- Condition $B_0 \bmod (A_0 \bmod B_0) \neq 0$ should not be considered as path condition is solely determined by c at this point.

c) Transformation performed in (a) changed behaviour of `gcd` for some inputs. Provide positive values for a and b such that `gcd(a, b)` and `bounded_gcd(a, b)` return different results. What does this mean for the output of the symbolic execution? Is it still an under-approximation of the result computed by `gcd`?

Answer After symbolic execution, we only need to look at $B_0 \bmod (A_0 \bmod B_0) \neq 0$

In `bounded_gcd(a, b)`, this condition is irrelevant due to the bound but in real `gcd`, the loop may continue if this condition holds true.

$A_0 = 2$ and $B_0 = 3$ satisfies this condition: $3 \bmod (2 \bmod 3) = 1 \neq 0$. Therefore, `bounded_gcd` will return 2 but `gcd` will return 1.

Because `bounded_gcd` returns a result that `gcd` would never return for the same set of inputs, it is not a valid under-approximation. A valid under-approximation would ensure that they behave identical for the same set of inputs. In other words, it needs to be a subset of all execution paths of `gcd` to be an under-approximation. So all bugs found in `bounded_gcd` can also occur in `gcd` but not vice versa. (Otherwise you cannot use `bounded_gcd` as a proxy for `gcd`!)

d) How can you modify transformation from (a) and the semantics of symbolic execution such that it computes an under-approximation of `gcd`? *Hint*: Try to rule out executions not possible in `gcd` using path constraints.

Answer The problem we have in `bounded_gcd` is that we break the loop after 2 iterations at maximum, which leads to incorrect results for some inputs, because the case `c=2` artificially terminates the loop. We need to identify these cases and rule them out.

We can look at the value of `b` when we break the loop, and determine if it artificially terminated the loop. If $b \neq 0$ we can know this was the case:

```
// requires: a > 0 AND b > 0
int bounded_gcd(int a, int b) {
    int c = 0;
    while (b != 0 && c < 2) {
        int tmp = b;
        b = a mod b;
        a = tmp;
        c = c + 1;
    }
    // Artificially terminated loop, rule out this case
    if (b != 0) {
        return -1; // Not possible to reach with gcd
    }
    return a;
}
```

Task 2 - Concolic Execution

Consider the two functions below. The function `my_pow`, written by Bob, is supposed to compute the power b^e for non-negative e . Bob's coworker Alice uses `my_pow` in the `pow_client` method.

Note that mathematically, for any $b \in \mathbb{Z}$ and even $e \in \mathbb{N}_{\text{even}}$, $b^e \geq 0$. Alice added an according assertion to `pow_client`.

```
int pow_client(int b, int e) {
    int r = my_pow(b, e);
    if (e mod 2 == 0) {
        if (r < 0) { // (*)
            assert(false); // should not happen
        }
    }
    return r;
}

int my_pow(int b, int e) {
    int r = b;
    for (int i = 1; i < e; i++) {
        r = r * b;
    }
    return r;
}

// incorrect implementation
```

Unfortunately, Bob's implementation of `my_pow` contains a bug, which may lead to a violation of Alice's assertion. In this task, you will use concolic execution to derive a test input for `pow_client` which spots the bug.

- a) Perform a first run of `pow_client` using concolic execution with concrete inputs $b = e = 0$. What is the path constraint gathered during the execution?

Answer

State 0 `int pow_client(int b, int e){`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 1 `int r = my_pow(b, e); → int my_pow(int b, int e){` (Looks weird but I think of myself as a debugger now)

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 2: `int r = b;`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π :

State 3: `for (int i = 1; i < e; i++){`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
 - $i \rightarrow 1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
 - $i \rightarrow 1$
- π :
 - $i \geq E_0$
 - Holds always true do to concrete $E_0 = 0$

State 4: `int r = my_pow(b, e)` (We saw it in State 1 but now `r` exists!)

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π :

State 5: if (e mod 2 == 0){

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π :
 - $E_0 \bmod 2 = 0$
 - Holds due to concrete values.
 - Return r as 0

- b) The previous run entered the (empty) **else** branch of the if-statement (*). Modify the path constraint from before by negating the sub-constraint collected for (*). Then, find a satisfying assignment for the new constraint.

Answer

The path constraint from before:

$$\pi : (1 \geq E_0) \wedge (E_0 \bmod 2 == 0) \wedge (B_0 \geq 0)$$

Negating the constraint (*):

- $\pi : (1 \geq E_0) \wedge (E_0 \bmod 2 == 0) \wedge (B_0 < 0)$

Assignment satisfying the new path constraint:

- $B_0 = -1$
- $E_0 = 0$

- c) Perform a second run using your new inputs from (b). What is the path constraint gathered during the execution? Can you reach the assertion?

Answer

State 0 int pow_client(int b, int e){

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 1 int r = my_pow(b,e)-> int my_pow(int b, int e){

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 2 `int r = b`

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π :

State 3 `for (int i=1; i<e; i++){`

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
 - $i \rightarrow 1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
 - $i \rightarrow 1$
- $\pi : 1 \geq E_0$
- Condition fulfilled

State 4 `r = my_pow(b, e);`

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π :

State 5 `if (e mod 2 == 0){`

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- $\pi : (1 \geq E_0) \wedge (E_0 \bmod 2 == 0)$
- Condition fulfilled, continue

State 5 `if (r < 0){`

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- $\pi : (1 \geq E_0) \wedge (E_0 \bmod 2 == 0) \wedge (B_0 < 0)$

State 6 `assert(false)` (done)

- γ :
 - $b \rightarrow -1$
 - $e \rightarrow 0$
 - $r \rightarrow -1$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow B_0$
- π : $(1 \geq E_0) \wedge (E_0 \bmod 2 == 0) \wedge (B_0 < 0)$

Final path constraint: $(1 \geq E_0) \wedge (E_0 \bmod 2 == 0) \wedge (B_0 < 0)$

Yes, we can reach the assertion

- d) Now, assume that the function `my_pow` is part of a library whose source code is not accessible for Alice. This is, she can only execute `my_pow` for concrete inputs. Repeat subtask (a) for this setting.

Answer

State 0 `int pow_client(int b, int e){`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 1 `int r = my_pow(b, e)-> int my_pow(int b, int e){`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
- π :

State 2 `int r = b;`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow R_0$
- π :

State 3 `if (e mod 2 == 0){`

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow R_0$
- π : $E_0 \bmod 2 == 0$

State 4 **if** ($r < 0$) {

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow R_0$
- $\pi : (E_0 \bmod 2 == 0) \wedge (R_0 \geq 0)$

State 5 **return** r ;

- γ :
 - $b \rightarrow 0$
 - $e \rightarrow 0$
 - $r \rightarrow 0$
- σ :
 - $b \rightarrow B_0$
 - $e \rightarrow E_0$
 - $r \rightarrow R_0$
- $\pi : (E_0 \bmod 2 == 0) \wedge (R_0 \geq 0)$

- e) Can you proceed analogously as in subtasks (b–c) to find inputs violating the assertion? Will concolic execution ever reach the assertion in this example?

Answer

No, we cannot proceed analogously as in subtasks (b–c) to find inputs to reach the assertion. In subtasks (b–c), we were able to negate the path constraint $B_0 < 0$ resulted from $r < 0$. But this is not possible in this case because we have $R_0 < 0$ instead of $B_0 < 0$ and we don't have control over R_0 as `my_pow` is a black-box function

Concolic execution may or may not reach the assertion depending on the concrete inputs. But it will never systematically reach the assertion in this example because SMT solver can't find the correct inputs to satisfy $R_0 < 0$.

- f) The symbolic execution engine KLEE can be used to perform concolic execution of C code. Use KLEE to generate a test case that exposes the bug from `my_pow`, using the SMT solver Z3 for solving the path conditions. How many paths does KLEE explore before finding the error? Which are the corresponding test cases produced by KLEE for each of these paths?

Answer

Using the `klee/klee:3.0` image, I ran the following command to generate test cases for `pow_client`:

```
clang -I /home/klee/klee_src/include/ -emit-llvm -c -g -O0 -Xclang
      -disable-O0-optnone pow.c -o pow.bc
```

```
klee -solver-backend=z3 -exit-on-error pow.bc 2>&1 | tee klee-output.txt
```

KLEE found the error in the first test case and stopped exploring further as the flag `-exit-on-error` was used. The concrete test case generated by KLEE was:

```
object 0: name: 'b'
object 0: int : -1

object 1: name: 'e'
object 1: int : 0
```

which corresponds to the call `pow_client(-1, 0)`. The detailed outputs of KLEE can be found in `klee-results`